Desenvolvimento de Sistemas Web II

Serviço de CRUD usando Ktor e Quarkus

DIM0547 — Turma 01 · Sprint 1 · 14/09

Prof. Fernando · UFRN · 2026.2

Roteiro da semana

Segunda, 14/09 — Um serviço, dois stacks

Bloco	O que vemos
Uma API do zero	Tarefas: servidor, JSON, POST, repositório
Ktor × Quarkus	Desenvolvendo o mesmo serviço nos dois
Corrotinas · DSL · Koin × CDI	Comparando abordagens

Quarta, 16/09 — Acompanhamento online · 🚀 **Entrega da Sprint 0, 23:59**

Segunda, 21/09 — Persistência, migrações e testes (Sprint 1, parte 2)

Onde paramos

Na Sprint 0 fechamos a fronteira HTTP e a arquitetura:

```
Recurso, método, status e erro em problem+json
Cache: Cache-Control, ETag e o 304
REST: restrições de Fielding, nível 2 de Richardson
Arquitetura limpa: a regra de dependência
```

O exemplo de hoje: Tarefas

Uma API simples, construída em 5 passos — cada um roda:

Passo	O que entra
1	Servidor "no ar"
2	Modelo `Tarefa` + `GET /tarefas` em JSON
3	`POST /tarefas`
4	Repositório por **interface** (regra de dependência)
5	Injeção de dependência (Koin × CDI)

Código pronto para consulta: `exemplos/ktor-tarefas/` e `exemplos/quarkus-tarefas/` (ver `PASSOS.md`).

Passo 1

Subindo o servidor

Passo 1 — servidor no ar

Ktor · `Aplicacao.kt`

```kotlin
fun main() {
  embeddedServer(CIO, port = 8080) {
    routing {
      get("/tarefas") {
        call.respondText("no ar")
      }
    }
  }.start(wait = true)
}
```

`./gradlew run`

Quarkus · `RecursoDeTarefas.java`

```java
@Path("/tarefas")
public class RecursoDeTarefas {
  @GET
  public String listar() {
    return "no ar";
  }
}
```

`mvn quarkus:dev`

Rotas como código (DSL) × **rotas como anotações**. Os dois expõem `/tarefas`; muda só onde o caminho é declarado — no `get` (Ktor) ou no `@Path` (Quarkus).

Passo 1 — configuração: código × convenção

Ktor — configura em **código**

```
embeddedServer(
  factory = CIO,    // o engine
  port = 8080,      // argumento nomeado
) {
  // trailing lambda: a config
}
```

Porta, engine e host: tudo em Kotlin.

Quarkus — por **convenção**

```
# application.properties
quarkus.http.port=8081
```

Sem código: o framework lê o arquivo.

> **Kotlin para quem conhece Java:** `port = 8080` é um **argumento nomeado**; o `{ ... }` no fim é uma **trailing lambda** — o último parâmetro, que é uma função, sai dos parênteses.

📖 **Ref.** Ktor — configuration in code · Quarkus — configuration

Passo 1 — o que é aquele `{ }`?

O `{ ... }` depois do `)` **é** o último argumento da função — uma lambda escrita **fora** dos parênteses. As duas formas são **idênticas**:

Antes — lambda dentro dos `( )`

```
embeddedServer(
  CIO,
  port = 8080,
  module = { routing { /* ... */ } },
)
```

Depois — *trailing lambda*

```
embeddedServer(CIO, port = 8080) {
  routing { /* ... */ }
}
```

A regra: se o **último** parâmetro é uma função, a lambda pode sair dos parênteses. Se for o **único** argumento, os `()` somem — por isso `routing { }` e `install(Koin) { }` parecem palavras da linguagem.

> **Para quem vem de Java:** não há equivalente — a lambda fica sempre dentro dos `( )`.

📖 **Ref.** Kotlin — passing trailing lambdas

Passo 2

Modelo e `GET` em JSON

Passo 2 — modelo e lista

Ktor

```kotlin
@Serializable
data class Tarefa(
  val id: Int,
  val titulo: String,
  val feita: Boolean = false,
)

install(ContentNegotiation) { json() }

get("/tarefas") {
  call.respond(tarefas)   // vira JSON
}
```

Quarkus

```java
public record Tarefa(
  int id,
  String titulo,
  boolean feita
) {}

// dependência: quarkus-rest-jackson

@GET
public List<Tarefa> listar() {
  return tarefas;          // vira JSON
}
```

`data class` ≈ `record`. `@Serializable` + `ContentNegotiation` ≈ Jackson: devolver o objeto **já é** devolver JSON.

Passo 2 — como o objeto vira JSON

Ktor

- `@Serializable` gera o serializador em **tempo de compilação** (plugin kotlinx.serialization) — **sem reflexão**
- `install(ContentNegotiation) { json() }` liga o formato JSON

Quarkus

- `quarkus-rest-jackson` serializa com **Jackson**
- Devolver o objeto de um método `@GET` já produz JSON — por convenção

> **Kotlin para quem conhece Java:** `data class` gera `equals` / `hashCode` / `toString` (como um `record`) e ainda um `copy()`. O `@Serializable` é processado por um **plugin do compilador**, não em runtime.

📖 **Ref.** kotlinx.serialization · Ktor — content negotiation · Quarkus — JSON REST

Passo 2 — a mesma resposta

```
curl localhost:8080/tarefas     # Ktor
curl localhost:8081/tarefas     # Quarkus
```

```
[
  { "id": 1, "titulo": "Estudar Ktor",    "feita": false },
  { "id": 2, "titulo": "Estudar Quarkus", "feita": false }
]
```

Passo 3

Criar com `POST`

Passo 3 — `POST /tarefas`

Ktor

```
@Serializable
data class NovaTarefa(val titulo: String)

post("/tarefas") {
  val nova = call.receive<NovaTarefa>()
  val criada = repositorio.adicionar(nova)
  call.respond(
    HttpStatusCode.Created, criada
  )
}
```

Quarkus

```
public record NovaTarefa(String titulo) {}

@POST
public Response criar(NovaTarefa nova) {
  Tarefa criada =
      repositorio.adicionar(nova);
  return Response
      .status(Status.CREATED)
      .entity(criada).build();
}
```

```
curl -X POST localhost:8080/tarefas -H 'Content-Type: application/json' \
     -d '{"titulo":"Escrever teste"}'
```

`receive<T>()` × parâmetro tipado: os dois desserializam o corpo. O **201** é explícito nos dois.

Passo 3 — desserializar o corpo

Ktor

```
val nova = call.receive<NovaTarefa>()
```

O tipo entre `< >` sobrevive em runtime (`inline` + `reified`), então o Ktor sabe **para qual classe** desserializar.

Quarkus

```
public Response criar(NovaTarefa nova)
```

O **tipo do parâmetro** já diz a classe; o framework desserializa antes de chamar o método.

> **Kotlin para quem conhece Java:** em Java os genéricos sofrem *type erasure* — o tipo some em runtime. Em Kotlin, `inline fun <reified T>` **mantém** o tipo; é o que faz `receive<T>()` funcionar sem passar `NovaTarefa.class`.

📖 **Ref.** Ktor — requests · Kotlin — reified type parameters

Entendendo: *type erasure* e `reified`

Por que o Ktor precisa de um truque para saber o tipo? Por causa de como a **JVM** trata genéricos.

`Java` — o tipo **some** em runtime

```
<T> T ler(String json) {
  // aqui NÃO dá para saber quem é T
  // (foi apagado: type erasure)
}

// por isso Jackson pede a classe:
mapper.readValue(json, NovaTarefa.class);
```

`Kotlin` — o tipo **fica**

```
inline fun <reified T> ler(json: String): T {
  // aqui T é conhecido: T::class funciona
}

// por isso o Ktor dispensa a classe:
call.receive<NovaTarefa>()
```

- **type erasure** (Java e a JVM): o argumento de tipo (`<T>`) existe só na **compilação**; em runtime ele some. Por isso as APIs Java pedem um `Class<T>` — o `.class`.
- **`inline`**: o compilador **copia** o corpo da função para o local da chamada.
- **`reified`**: com a função `inline`, o compilador **cola ali o tipo real**. Aí `T` é conhecido em runtime, e `T::class` funciona.

📖 **Ref.** Kotlin — type erasure · Kotlin — reified

Como o `inline` faz a "colagem"

A função de desserializar, definida **uma vez**:

```
inline fun <reified T> receive(): T =
    parse(corpo, T::class)   // T::class só é possível por causa do reified
```

No **local da chamada**, o compilador cola o corpo e troca `T` pelo tipo real:

```
  Você escreve:
      val n = call.receive<NovaTarefa>()

                  │  inline  → cola o corpo da função aqui
                  │  reified → troca T por NovaTarefa
                  ▼

  O compilador gera (no mesmo lugar):
      val n = parse(corpo, NovaTarefa::class)
```

- Uma **cópia por tipo**: `receive<NovaTarefa>()` e `receive<Tarefa>()` geram dois corpos, cada um com o seu `::class`.
- Por isso `inline` é para funções **pequenas** — o corpo é duplicado em cada chamada.

> Em Java, `T` sumiria e você passaria `NovaTarefa.class` manualmente. Em Kotlin, o compilador escreve isso por você, em cada ponto de uso.

📖 **Ref.** Kotlin — inline functions

Passo 3 — responder com o status certo

Ktor

```
call.respond(
  HttpStatusCode.Created,   // 201
  criada,
)
```

Status e corpo numa só chamada.

Quarkus

```
return Response
    .status(Status.CREATED)  // 201
    .entity(criada)
    .build();
```

Um *builder* do Jakarta REST monta a resposta.

> O **201 Created** é **explícito** nos dois — muda a forma: uma chamada (Ktor) × encadeamento de *builder* (Quarkus). Devolver só o objeto daria **200 OK**.

📖 **Ref.** Ktor — responses · Jakarta REST — `Response`

Passo 4

O repositório por trás de uma interface

Passo 4 — a regra de dependência

A rota não deve saber **de onde** vêm os dados. Ela fala com uma **interface**.

Ktor

```
interface RepositorioDeTarefas {
  fun listar(): List<Tarefa>
  fun adicionar(n: NovaTarefa): Tarefa
}

class RepositorioEmMemoria
  : RepositorioDeTarefas { /* ... */ }
```

Quarkus

```
public interface RepositorioDeTarefas {
  List<Tarefa> listar();
  Tarefa adicionar(NovaTarefa n);
}

class RepositorioEmMemoria
  implements RepositorioDeTarefas { }
```

Na **parte 2 (21/09)** trocamos memória por **Postgres** — e as rotas não mudam, porque dependem da interface.

Passo 5

Injeção de dependência

Passo 5 — Koin × CDI

Ktor + Koin — grafo explícito

```
install(Koin) {
  modules(module {
    single<RepositorioDeTarefas> {
      RepositorioEmMemoria()
    }
  })
}

val repositorio
  by inject<RepositorioDeTarefas>()
```

Quarkus + CDI — por anotação

```
@ApplicationScoped
public class RepositorioEmMemoria
    implements RepositorioDeTarefas { }

// no recurso:
@Inject
RepositorioDeTarefas repositorio;
```

Mesma ideia: **quem usa não constrói**. Koin reúne a montagem num lugar (código); o CDI a espalha em anotações, resolvidas pelo contêiner.

CDI — o que é?

CDI = *Contexts and Dependency Injection*, o padrão de injeção do **Jakarta EE** (o Quarkus implementa).

- O **contêiner** cria, guarda e injeta os objetos (*beans*) — você não dá `new`
- `@ApplicationScoped`: **um** bean para toda a aplicação (há `@RequestScoped`, etc.)
- `@Inject`: "me entregue esse bean" — o contêiner resolve

```
@ApplicationScoped
public class RepositorioEmMemoria implements RepositorioDeTarefas { }

@Inject RepositorioDeTarefas repositorio;   // o contêiner injeta
```

> Para quem já viu Spring: é a mesma ideia do `@Autowired`. O Koin faz esse papel no Ktor — mas **em código**, não por anotação.

📖 **Ref.** Quarkus — CDI · Jakarta CDI

Koin — injeção como DSL

No Ktor a injeção é uma **biblioteca** (Koin), escrita em **código Kotlin**:

```
install(Koin) {
  modules(module {
    single<RepositorioDeTarefas> { RepositorioEmMemoria() }  // como construir
  })
}

val repositorio by inject<RepositorioDeTarefas>()           // como obter
```

- `single { }` — **um** objeto compartilhado (equivale ao `@ApplicationScoped`)
- `by inject()` — *property delegate*: entrega o objeto quando ele é usado

> **Código × anotação.** Koin reúne a montagem num lugar, visível e depurável como código; o CDI a espalha em anotações resolvidas pelo contêiner. Ambos têm o mesmo objetivo.

📖 **Ref.** Koin com Ktor · Kotlin — delegated properties

Entendendo: propriedades delegadas (`by`)

A palavra `by` diz: "as leituras (e escritas) desta propriedade são **delegadas** a outro objeto".

```
val repositorio by inject<RepositorioDeTarefas>()
//             ▲               ▲
//     a propriedade      o "delegate" (Koin): sabe entregar o objeto
```

- Ler `repositorio` chama o `getValue()` do *delegate* — aqui o Koin resolve e devolve o bean.
- O objeto é buscado **quando usado** (preguiçoso), não na declaração.
- Outros *delegates* comuns: `by lazy { }` (calcula uma vez) e, no Compose, `by remember { }`.

> **Kotlin para quem conhece Java:** não há equivalente direto. Em Java você escreveria um *getter* (`getRepositorio()`) e o chamaria.

📖 **Ref.** Kotlin — delegated properties

Corrotinas

`suspend` marca uma função que pode **pausar** num *ponto de suspensão* — a chamada a outra função `suspend`, como uma consulta ao banco — e **retomar** depois, liberando a thread enquanto espera. O handler do Ktor já é `suspend`.

```
// uma operação de E/S é marcada com suspend:
suspend fun buscarNoBanco(): List<Tarefa> { /* ... */ }

get("/tarefas") {              // o handler já é suspend
  val dados = buscarNoBanco()  // ponto de suspensão: pausa aqui,
  call.respond(dados)          // devolve a thread e retoma quando o banco responde
}
```

No Quarkus, o framework administra o modelo de execução.

> São dois jeitos de não desperdiçar threads esperando E/S. Um é explícito, e o outro transparente.

Bloqueante vs Não bloqueante

No modelo clássico (uma thread por requisição), esperar E/S — banco, rede — deixa a thread parada. Muitas esperas, muitas threads presas: não escala.

```
Bloqueante          a thread fica presa esperando o banco
  Req A → [thread 1] ──────(parada)──────► responde

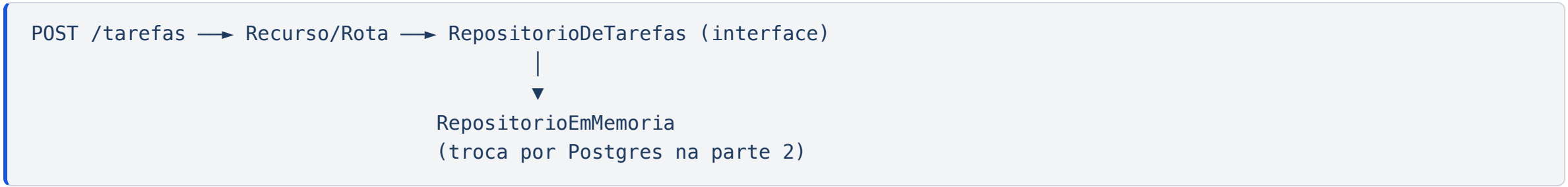
Não bloqueante      a thread é liberada e atende outra
  Req A → [thread 1] ──agenda a E/S──► atende Req B
          (banco respondeu) ──► retoma Req A
```

- **Ktor** — `suspend` **pausa** a corrotina na espera e **devolve** a thread; retoma quando a E/S termina. O Ktor já chama seu handler como `suspend`.
- **Quarkus** — um **event loop** (Vert.x/Netty) faz o mesmo. O framework administra o modelo: você escreve código imperativo comum e poucas threads servem muitas requisições.

> **Para quem conhece o modelo Servlet (uma thread por requisição):** os dois evitam prender a thread em E/S. Ktor é **explícito** (`suspend`); Quarkus é **transparente** — o runtime decide, e você marca `@Blocking` só quando quiser bloquear de propósito.

📖 **Ref.** Quarkus — reactive architecture · Kotlin — coroutines

Padrão repositório



- A **rota/recurso** é o que um desenvolvedor precisa saber para usar a API.
- No entanto, o **repositório** é que guarda as regras de acesso aos dados.
- A **interface** desacopla um do outro através de injeção de dependência.

Padrão repositório — a ideia

Repositório é um *design pattern*: uma interface que acessa os dados **como uma coleção em memória**, escondendo *onde* e *como* eles são persistidos (memória, SQL, API…).

- **Intenção** — separar a **regra de negócio** do **mecanismo de persistência** (Fowler: um *mediador entre o domínio e o mapeamento de dados*).
- **Arquitetura (regra de dependência)** — a **interface** é definida no domínio; a **implementação** mora na infraestrutura. A dependência aponta para dentro: o banco é um **detalhe**.
- **Ganho concreto** — trocar `EmMemoria` por `Postgres` (parte 2) e **testar** com um repositório falso, sem tocar nas rotas.

📖 **Ref.** Fowler — *Repository* (EN) · Microsoft Learn — *O padrão de repositório* (PT)

Como rodar (na aula e em casa)

`ktor-tarefas/`

```
cd exemplos/ktor-tarefas
./gradlew run
# http://localhost:8080
```

`quarkus-tarefas/`

```
cd exemplos/quarkus-tarefas
mvn quarkus:dev
# http://localhost:8081
```

Os dois sobem lado a lado. Bata a **mesma** requisição nos dois e compare:

```
curl localhost:8080/tarefas
curl localhost:8081/tarefas
```

Para a próxima aula (21/09)

Na parte 2 da Sprint 1, o repositório em memória vira banco de verdade:

- **Persistência e migrações**: Exposed (Kotlin) × Hibernate/Panache (Java), **Flyway** em ambos
- **Testes** com **Testcontainers**
- **Documentação**: OpenAPI (`ktor-openapi` × `smallrye-openapi`)

E **16/09** é acompanhamento online — tragam o projeto rodando e as dúvidas. 🚀 Entrega da Sprint 0 nesse dia, 23:59.

Onde estudar depois

Fonte	Foco
`ktor.io/docs` · *Creating a new project*	Ktor do zero, engine e plugins
`quarkus.io/guides/getting-started`	Quarkus REST, CDI e dev mode
`start.ktor.io` · `code.quarkus.io`	Geradores oficiais de projeto
MUSI · `api-ktor/` e `api-quarkus/`	As mesmas peças, em escala